

Тема 7. Поток Stream в Node.js

Readable, Writable, pipe(), backpressure и pipeline()

Цель занятия

- Научиться обрабатывать большие объёмы данных по частям, не загружая весь файл или весь ответ в оперативную память.
- Понять, зачем нужны потоки в Node.js.
- Разобрать основные типы потоков: Readable, Writable, Duplex, Transform.
- Научиться читать файл частями через `fs.createReadStream()`.
- Научиться записывать данные через `fs.createWriteStream()`.
- Использовать `pipe()` для соединения потоков.
- Понять, когда лучше использовать `stream.pipeline()`.

Учебные вопросы

- 1. Концепция потоков: зачем они нужны и какие бывают типы потоков.
- 2. Readable Stream: как получать данные по частям.
- 3. Writable Stream: как отправлять данные по частям.
- 4. Метод `pipe()`: соединяем поток чтения с потоком записи.
- 5. `stream.pipeline()`: безопасная цепочка потоков с обработкой ошибок.
- 6. Практическое применение: передача больших файлов и streaming media.



1. Концепция потоков

Почему не всегда нужно читать весь файл сразу

Обычное чтение файла и потоковый подход

`fs.readFile()`

- Асинхронно читает файл.
- В итоге собирает содержимое файла целиком в памяти.
- Для больших файлов может резко увеличить потребление RAM.
- Подходит для небольших файлов и простых задач.

`fs.createReadStream()`

- Читает данные порциями.
- Не держит весь файл в памяти сразу.
- Позволяет начать обработку до полного чтения файла.
- Подходит для больших файлов и передачи по сети.

Почему большие файлы опасно читать целиком

- Попытка целиком загрузить очень большой файл может привести к чрезмерному потреблению памяти и завершению процесса с ошибкой.
- Проблема не в том, что `async readFile()` блокирует весь сервер на время чтения.
- Главная проблема — итоговый Buffer с содержимым файла должен поместиться в памяти.
- Чем больше файл, тем выше риск нехватки памяти.
- Для потоковой обработки лучше использовать `createReadStream()`.

Что такое поток Stream

- Stream — это абстрактный интерфейс для работы с данными по частям, которые называются chunks.
- Поток не является способом хранения данных.
- Поток является способом постепенной доставки данных.
- Приложение получает очередную порцию данных и сразу может её обработать.
- Память расходуется порциями и обычно не растёт линейно от размера всего файла.
- Расход памяти зависит от highWaterMark, числа потоков, Transform-этапов и логики приложения.

Главная идея Streams

- Не жди весь файл — работай с тем, что уже пришло.
- Файл, сетевой запрос или ответ можно обрабатывать постепенно.
- Обработка начинается после получения первой порции данных.
- Streams особенно полезны для файлов, видео, архивов, сетевых соединений и проксирования данных.
- При правильном использовании уменьшается риск перегрузки памяти.

Типы потоков в Node.js

- В Node.js есть четыре основных типа потоков.
- Readable — источник данных.
- Writable — место назначения для данных.
- Duplex — одновременно Readable и Writable.
- Transform — Duplex-поток, который изменяет данные на лету.
- Потоки работают через события, потому что являются EventEmitter-объектами.

Четыре типа потоков

Тип	Назначение	Примеры
Readable	Источник, из которого читают данные	<code>fs.createReadStream()</code> , <code>process.stdin</code> , <code>req</code>
Writable	Приёмник, в который записывают данные	<code>fs.createWriteStream()</code> , <code>process.stdout</code> , <code>res</code>
Duplex	Чтение и запись одновременно	<code>net.Socket</code>
Transform	Изменение данных на лету	<code>zlib.createGzip()</code> , <code>crypto streams</code>

Readable Stream

- Readable — это поток, из которого приложение получает данные.
- `fs.createReadStream()` — чтение файла.
- `process.stdin` — ввод из терминала.
- `req` в HTTP-сервере — входящий HTTP-запрос.
- Ключевое событие: `data`.
- `data` приносит очередную порцию данных `chunk`.

Writable Stream

- Writable — это базовый тип потока записи, в который приложение отправляет данные.
- fs.WriteStream — файловая реализация Writable.
- process.stdout — вывод в консоль.
- res в HTTP-сервере — HTTP-ответ.
- Ключевой метод: write().
- Завершение записи выполняется через end().

Duplex Stream

- Duplex — поток, который одновременно является Readable и Writable.
- У Duplex есть сторона чтения и сторона записи.
- Вход и выход могут быть независимыми.
- Пример: `net.Socket`.
- Через `socket` можно одновременно отправлять и получать данные.

Transform Stream

- Transform — разновидность Duplex-потока, который изменяет данные в процессе передачи.
- Данные входят в поток.
- Поток преобразует данные.
- На выходе появляются уже изменённые данные.
- Пример: `zlib.createGzip()` сжимает данные.
- Пример: `crypto stream` может шифровать данные.

Buffer vs Stream

Характеристика	Buffer / readFile()	Stream / createReadStream()
Память	Содержимое файла собирается целиком	В памяти находятся ограниченные порции данных
Старт обработки	После завершения чтения	После получения первой порции данных
Большие файлы	Может резко увеличить расход RAM	Не требует загрузки всего файла сразу
Буферизация	Зависит от размера данных	Определяется highWaterMark и типом потока

Итог по концепции потоков

1

Stream — это способ транспортировки данных, а не способ их хранения.

2

Потоки позволяют работать с данными по частям.

3

Readable отдаёт данные, **Writable** принимает данные.

4

Transform изменяет данные по пути.

5

Потоки особенно полезны для больших файлов и сетевых данных.

6

Не нужно утверждать, что **stream** всегда занимает ровно 64 KiB RAM: реальный расход зависит от буферов и логики приложения.



2. Readable Stream

Как получать данные по частям

Что такое chunk

- Chunk — это очередная порция данных, которую поток передал потребителю.
- Node.js не обязан отдавать файл по одному символу или одной строке.
- Данные приходят порциями.
- У `fs.createReadStream()` `highWaterMark` по умолчанию равен 64 KiB.
- Реальный размер chunk может быть меньше `highWaterMark`.
- Для текстового чтения можно указать `encoding: 'utf8'`.

Два режима Readable Stream

- Paused — данные не текут автоматически; их нужно запрашивать вручную.
- Flowing — данные передаются максимально быстро, как только доступны.
- Flowing mode включается, если добавить обработчик события data.
- Метод `pipe()` тоже переводит поток в режим передачи данных.
- `resume()` явно запускает поток.

Основные события Readable Stream

Событие	Когда срабатывает	Что передаёт
data	Поток передал очередной chunk	chunk
end	Источник больше не отдаёт данные	—
error	Произошла ошибка чтения	Error object
close	Поток был закрыт	—

Событие `open` у файлового потока

- `open` — событие не универсального `Readable`, а файлового `fs.ReadStream`.
- `open` срабатывает, когда файл успешно открыт.
- В `callback` может передаваться `file descriptor`.
- Это событие относится именно к файловым потокам.
- Для общего объяснения `Readable Stream` важнее `data`, `end`, `error` и `close`.

Пример fs.createReadStream()

JAVASCRIPT

```
import fs from 'node:fs';
import path from 'node:path';

const stream = fs.createReadStream(
  path.join(import.meta.dirname, 'war_and_peace.txt'),
  {
    encoding: 'utf8',
    highWaterMark: 16 * 1024
  }
);

stream.on('data', (chunk) => {
  console.log('--- Новый chunk ---');
  console.log(chunk.length); // длина полученной строки
  console.log(chunk);
});
```

Что важно

- Пример использует ES Modules: import вместо require().
- import.meta.dirname заменяет __dirname в ESM-проекте.
- encoding: 'utf8' возвращает строку, а не Buffer.
- chunk.length для строки — это длина строки в JavaScript, а не гарантированное число визуальных символов.

Метод `fs.createReadStream()`

- `fs.createReadStream()` создаёт объект `Readable Stream`, связанный с конкретным файлом.
- Метод принимает путь к файлу.
- Вторым аргументом можно передать объект настроек.
- Метод не загружает весь файл сразу.
- Чтение начинается при подписке на `data`, вызове `pipe()` или `resume()`.
- Для кроссплатформенной сборки пути удобно использовать `path.join()`.

Основные options для createReadStream()

Option	Назначение	Комментарий
encoding	Декодирование bytes в строку	Например: 'utf8'
highWaterMark	Порог внутренней буферизации	Для fs.createReadStream() default 64 KiB
start	Начальный байт	Отсчёт с 0
end	Последний байт	Включительно

Что возвращает `createReadStream()`

- Метод возвращает объект `fs.ReadStream`.
- Это файловая реализация `Readable Stream`.
- Объект не содержит весь файл сразу.
- Он умеет постепенно читать файл и передавать `chunks`.
- Чтение можно контролировать методами `pause()`, `resume()`, `destroy()`, `pipe()`.

Методы управления Readable Stream

Метод	Назначение
<code>pause()</code>	Приостанавливает поток данных
<code>resume()</code>	Возобновляет поток данных
<code>destroy()</code>	Закрывает поток и освобождает ресурс
<code>pipe(destination)</code>	Передаёт данные в Writable-поток

Метод `on()`

- `on()` используется для подписки на события потока.
- Первый аргумент — имя события.
- Второй аргумент — callback-функция.
- Callback срабатывает, когда событие произошло.
- Потоки используют события, потому что Node.js работает асинхронно.

Разбор stream.on('data')

JAVASCRIPT

```
stream.on('data', (chunk) => {  
  // Этот код сработает много раз  
});
```

Что важно

- data — стандартное событие Readable Stream.
- Оно означает, что поток передал потребителю очередную порцию данных.
- chunk — аргумент, в который попадает текущая порция данных.
- data не означает, что буфер обязательно заполнился ровно на 64 KiB.

Почему data срабатывает много раз

- Если файл весит 1 MiB, а highWaterMark равен 64 KiB, событие data может сработать примерно 16 раз.
- Это учебное приближение, а не точная гарантия.
- Последний chunk может быть меньше.
- Реальный размер chunk зависит от источника данных, кодировки и внутренней буферизации.
- highWaterMark — это порог буферизации, а не обещанный размер каждого chunk.

Readable Stream: полный контроль

JAVASCRIPT

```
import fs from 'node:fs';
import path from 'node:path';

const stream = fs.createReadStream(
  path.join(import.meta.dirname, 'war_and_peace.txt'),
  {
    encoding: 'utf8',
    highWaterMark: 16 * 1024
  }
);

stream.on('data', (chunk) => {
  console.log('--- Новый chunk ---');
  console.log(chunk.length);
  console.log(chunk);
});

stream.on('end', () => {
  console.log('Чтение файла завершено');
});

stream.on('error', (err) => {
  console.error('Ошибка чтения:', err.message);
});
```

Что важно

- data может сработать много раз.
- end сработает один раз после окончания чтения.
- error нужен для обработки ошибок.
- Такой подход даёт полный ручной контроль.

Итог по Readable Stream

- 1 Readable Stream — источник данных.
- 2 `fs.createReadStream()` читает файл порциями.
- 3 `chunk` — очередная порция данных, а не обязательно ровно 64 KiB.
- 4 `highWaterMark` управляет порогом буферизации.
- 5 Событие `data` передаёт очередной `chunk`.
- 6 События `end` и `error` позволяют корректно завершать и диагностировать чтение.
- 7 В ESM-проекте используем `import` и `import.meta.dirname`.



3. Writable Stream

Как отправлять данные по частям

Что такое Writable Stream

- Writable — базовый тип потока, предназначенный для записи данных в место назначения.
- fs.WriteStream — файловая реализация Writable.
- process.stdout — Writable-поток вывода в консоль.
- HTTP response res — Writable-поток ответа клиенту.
- В Writable мы сами отправляем данные методом write().
- После завершения записи вызываем end().

Пример createWriteStream()

JAVASCRIPT

```
import fs from 'node:fs';
import path from 'node:path';

const writeStream = fs.createWriteStream(
  path.join(import.meta.dirname, 'writeTest.txt'),
  { encoding: 'utf8' }
);

writeStream.write('Hello\n');
writeStream.write('World\n');
writeStream.write('!!!!\n');

writeStream.end();
```

Что важно

- createWriteStream() создаёт поток записи в файл.
- write() можно вызвать несколько раз.
- end() сообщает, что данных больше не будет.
- После end() писать в этот поток уже нельзя.

Основные методы Writable Stream

Метод	Назначение
<code>write(chunk)</code>	Отправляет порцию данных в поток
<code>end([chunk])</code>	Завершает запись; optional chunk будет записан последним
<code>destroy()</code>	Немедленно закрывает поток и отменяет незавершённые операции
<code>on('drain')</code>	Позволяет дождаться освобождения внутреннего буфера

Основные события Writable Stream

Событие	Когда срабатывает
finish	После end(), когда все данные обработаны Writable-поток и переданы нижележащей системе
error	Если произошла ошибка записи
drain	Когда внутренний буфер потока снова готов принимать данные
close	Когда поток был закрыт

Writable Stream: запись строк

JAVASCRIPT

```
import fs from 'node:fs';
import path from 'node:path';

const writeStream = fs.createWriteStream(
  path.join(import.meta.dirname, 'output.txt'),
  { encoding: 'utf8' }
);

writeStream.write('Строка 1\n');
writeStream.write('Строка 2\n');
writeStream.write('Строка 3\n');

writeStream.end('Конец файла\n');
```

Что важно

- `write()` добавляет данные в поток.
- `end()` завершает запись.
- Если передать строку в `end()`, она будет записана последней.
- После `end()` поток не принимает новые `write()`.

События finish и error

JAVASCRIPT

```
writeStream.on('finish', () => {  
  console.log('Поток завершил запись всех данных');  
});  
  
writeStream.on('error', (err) => {  
  console.error('Ошибка записи:', err.message);  
});
```

Что важно

- finish не означает гарантированную физическую запись на носитель.
- finish означает, что Writable-поток завершил обработку всех данных после end().
- error нужен для обработки проблем с записью.
- Например, может не быть прав на запись или может закончиться место.

Важно запомнить: end и finish

- У Readable есть событие end: источник больше не отдаёт данные.
- У Writable есть событие finish: запись в поток завершена.
- end у Readable и finish у Writable — разные события.
- Не путайте окончание чтения и окончание записи.



4. Метод pipe()

Соединяем поток чтения с потоком записи

Проблема ручного копирования

- Если вручную читать через `data` и сразу писать через `write()`, можно столкнуться с разной скоростью чтения и записи.
- Например, чтение с SSD быстрее записи на медленный диск или отправки по сети.
- Если источник отдаёт данные быстрее, чем приёмник успевает их обработать, данные будут накапливаться в памяти.
- Нужен механизм, который учитывает скорость приёмника.
- Эта проблема называется `backpressure`.

Что такое pipe()

JAVASCRIPT

```
readStream.pipe(writeStream);
```

Что важно

- pipe() связывает Readable-поток с Writable-поток.
- Данные автоматически передаются из источника в приёмник.
- pipe() управляет backpressure.
- Когда источник заканчивается, pipe() по умолчанию завершает destination через end().

Почему используют `pipe()`

- Не нужно вручную реализовывать передачу данных через `data` и `write`.
- `pipe()` автоматически учитывает `backpressure`.
- Код становится короче и понятнее.
- Для простой передачи из точки А в точку Б `pipe()` подходит хорошо.
- Ошибки всё равно нельзя игнорировать: их нужно обрабатывать отдельно или использовать `pipeline()`.

Пример: копирование файла через pipe()

JAVASCRIPT

```
import fs from 'node:fs';
import path from 'node:path';

const src = fs.createReadStream(
  path.join(import.meta.dirname, 'video.mp4')
);

const dest = fs.createWriteStream(
  path.join(import.meta.dirname, 'video.copy.mp4')
);

src.pipe(dest);

dest.on('finish', () => {
  console.log('Копирование завершено');
});

src.on('error', (err) => {
  console.error('Ошибка чтения:', err.message);
});

dest.on('error', (err) => {
  console.error('Ошибка записи:', err.message);
});
```

Что важно

- src — поток чтения.
- dest — поток записи.
- pipe() передаёт данные и управляет скоростью.
- Для учебного примера добавлены error listeners на оба потока.

Цепочки потоков

- `pipe()` возвращает поток назначения, поэтому можно строить цепочки обработки.
- Источник читает данные.
- Transform-поток изменяет данные.
- Writable-поток сохраняет или отправляет результат.
- Пример: чтение файла → сжатие `gzip` → запись архива.

Пример: чтение → сжатие → запись

JAVASCRIPT

```
import fs from 'node:fs';
import path from 'node:path';
import zlib from 'node:zlib';

const readStream = fs.createReadStream(
  path.join(import.meta.dirname, 'war_and_peace.txt')
);

const gzip = zlib.createGzip();

const writeStream = fs.createWriteStream(
  path.join(import.meta.dirname, 'war_and_peace.txt.gz')
);

readStream
  .pipe(gzip)
  .pipe(writeStream);
```

Что важно

- readStream читает исходный файл.
- gzip является Transform-поток.
- writeStream записывает сжатый результат.
- Для production-кода лучше использовать pipeline() с обработкой ошибок.

5. stream.pipeline()

Безопасная цепочка потоков с обработкой ошибок

Зачем нужен `pipeline()`

- `pipeline()` соединяет потоки в цепочку и корректно обрабатывает ошибки и завершение.
- `pipe()` остаётся полезным API для простых случаев.
- Но `pipe()` не означает, что ошибки можно игнорировать.
- Если в середине цепочки возникает ошибка, нужно корректно остановить остальные потоки.
- `pipeline()` упрощает обработку ошибок и `cleanup`.
- Для цепочек чтение $\rightarrow$ `transform` $\rightarrow$ запись `pipeline()` обычно безопаснее.

Пример pipeline()

JAVASCRIPT

```
import fs from 'node:fs';
import zlib from 'node:zlib';
import { pipeline } from 'node:stream/promises';

await pipeline(
  fs.createReadStream('war_and_peace.txt'),
  zlib.createGzip(),
  fs.createWriteStream('war_and_peace.txt.gz')
);

console.log('Файл успешно архивирован');
```

Что важно

- pipeline() принимает несколько потоков по порядку.
- Если один поток выдаст ошибку, pipeline() завершится с ошибкой.
- Вариант из node:stream/promises удобно использовать с await.
- Для учебного ESM-кода такой пример согласуется с import/export.

pipe() и pipeline()

pipe()

- Короткий синтаксис.
- Хорош для простых цепочек.
- Управляет backpressure.
- Ошибки нужно обрабатывать отдельно.

pipeline()

- Подходит для production-цепочек.
- Упрощает обработку ошибок.
- Корректно завершает цепочку потоков.
- Удобен с async/await через `node:stream/promises`.



6. Практическое применение

Передача больших файлов и streaming media

Где применяются Streams

- Копирование больших файлов.
- Сжатие и распаковка данных.
- Передача файлов по HTTP.
- Проксирование сетевых данных.
- Streaming audio/video.
- Обработка логов и больших CSV/JSONL-файлов.

Потоковая передача файла по HTTP

JAVASCRIPT

```
import http from 'node:http';
import fs from 'node:fs';

http.createServer((req, res) => {
  res.writeHead(200, {
    'Content-Type': 'video/mp4'
  });

  fs.createReadStream('video.mp4')
    .pipe(res);

}).listen(3000);
```

Что важно

- `createReadStream()` читает файл по частям;
- `res` — Writable Stream;
- `.pipe(res)` отправляет chunks клиенту;
- весь файл не загружается в RAM.

Простой streaming файла через Express

JAVASCRIPT

```
import express from 'express';
import fs from 'node:fs';
import path from 'node:path';

const app = express();

app.get('/video', (req, res) => {
  const filePath = path.join(import.meta.dirname, 'video.mp4');

  res.setHeader('Content-Type', 'video/mp4');

  fs.createReadStream(filePath)
    .on('error', () => {
      res.status(500).end('Video streaming error');
    })
    .pipe(res);
});
```

Что важно

- Файл не загружается целиком в память.
- `createReadStream()` читает его порциями.
- `res` является Writable-потокотом HTTP-ответа.
- Это простой пример последовательной передачи файла.

Важное замечание про видео

- Для настоящего video streaming часто нужны Range requests.
- Простой `pipe(res)` передаёт файл последовательно.
- Для перемотки видео браузер обычно использует HTTP Range.
- Range позволяет запросить только часть файла.
- В этой лекции важно понять базовую идею: файл можно отдавать потоково.
- Подробная реализация Range — отдельная практическая тема.

Выводы

- 1 `fs.readFile()` асинхронен, но в итоге собирает весь файл в памяти.
- 2 `Streams` позволяют обрабатывать данные по частям.
- 3 `Readable` отдаёт `chunks`, `Writable` принимает `chunks`.
- 4 `highWaterMark` — порог буферизации, а не гарантированный размер каждого `chunk`.
- 5 `data` означает передачу очередной доступной порции данных.
- 6 `finish` у `Writable` не гарантирует физическую запись на диск.
- 7 `pipe()` соединяет потоки и управляет `backpressure`.
- 8 Ошибки при `pipe()` нужно обрабатывать отдельно.
- 9 `pipeline()` удобен для безопасных цепочек потоков.
- 10 В ESM-проектах используем `import`, `node:fs` и `import.meta.dirname`.

Контрольные вопросы

- 1 Почему `fs.readFile()` не подходит для очень больших файлов?
- 2 Что такое `chunk`?
- 3 Почему размер `chunk` не обязан быть равен `highWaterMark`?
- 4 Чем `Readable` отличается от `Writable`?
- 5 В чём разница между событиями `end` и `finish`?
- 6 Почему `src.pipe(dest)` лучше ручного `data/write` для простого копирования?
- 7 Почему ошибки при `pipe()` нельзя игнорировать?
- 8 Когда лучше использовать `pipeline()`?
- 9 Как в ESM заменить `require('fs')` и `__dirname`?
- 10 Как прочитать только часть файла через параметры `start` и `end`?